Métodos de Análisis de Datos 1

Clase 6: Normalización e Integración

Universidad Nacional del Sur

Contenidos de hoy

El problema de las escalas

Ejemplo

Edad	Ingreso
25	50000
30	65000
45	80000

El ingreso tiene valores mucho mayores que la edad

El problema de las escalas

Ejemplo

Edad	Ingreso
25	50000
30	65000
45	80000

El ingreso tiene valores mucho mayores que la edad

Problemas

- Algoritmos sensibles a escala (KNN, SVM, redes neuronales)
- Variables con mayor rango dominan el modelo
- Convergencia lenta en algoritmos de optimización
- Interpretación difícil de coeficientes

Solución: Escalar las variables

¿Qué es escalar?

Transformar variables a rangos comparables

Dos técnicas principales

1. **Normalización (Min-Max):** Escalar a rango [0, 1]
2. **Estandarización (Z-score):** Transformar a media 0 y desviación estándar 1

Solución: Escalar las variables

¿Qué es escalar?

Transformar variables a rangos comparables

Dos técnicas principales

1. **Normalización (Min-Max):** Escalar a rango [0, 1]
2. **Estandarización (Z-score):** Transformar a media 0 y desviación estándar 1

¿Cuándo escalar?

- KNN, SVM, redes neuronales
- Algoritmos de clustering (K-means)
- Regresión regularizada (Ridge, LASSO)
- Árboles de decisión, Random Forest

Min-Max Scaling (Normalización)

Fórmula

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Transforma valores al rango [0, 1]

Ejemplo

Variable: [10, 20, 30, 40, 50]

$$X_{norm}(10) = \frac{10 - 10}{50 - 10} = 0.00$$

$$X_{norm}(30) = \frac{30 - 10}{50 - 10} = 0.50$$

$$X_{norm}(50) = \frac{50 - 10}{50 - 10} = 1.00$$

Min-Max: Código

Python:

```python
from sklearn.preprocessing import
    MinMaxScaler

# Normalizar
scaler = MinMaxScaler()
df[['edad', 'ingreso']] = scaler.
    fit_transform(
     df[['edad', 'ingreso']]
)

# Rango personalizado [-1, 1]
scaler = MinMaxScaler(
     feature_range=(-1, 1)
)
df[['edad', 'ingreso']] = scaler.
    fit_transform(
```

R:

```r
# Manual
normalizar <- function(x) {
  (x - min(x, na.rm = TRUE)) /
  (max(x, na.rm = TRUE) - min(x,
      na.rm = TRUE))
}

df <- df %>%
  mutate(across(
    c(edad, ingreso),
    normalizar
  ))

# Con caret
library(caret)
preproc <- preProcess(
```

Standardization (Z-score)

Fórmula

$$X_{std} = \frac{X - \mu}{\sigma}$$

Transforma a media 0 y desviación estándar 1

Ejemplo

Variable: [10, 20, 30, 40, 50]
Media (μ) = 30, Desv. Est. (σ) = 14.14

$$X_{std}(10) = \frac{10 - 30}{14.14} = -1.41$$
$$X_{std}(30) = \frac{30 - 30}{14.14} = 0.00$$
$$X_{std}(50) = \frac{50 - 30}{14.14} = 1.41$$

Standardization: Código

Python:

```python
from sklearn.preprocessing import
    StandardScaler

scaler = StandardScaler()
df[['edad', 'ingreso']] = scaler.
    fit_transform(
     df[['edad', 'ingreso']]
)

# Verificar
print(df['edad'].mean())   # ~0
print(df['edad'].std())    # ~1
```

R:

```r
# Estandarizar
df <- df %>%
  mutate(across(
    c(edad, ingreso),
    scale
  ))

# Con caret
preproc <- preProcess(
  df[, c('edad', 'ingreso')],
  method = c("center", "scale")
)
df[, c('edad', 'ingreso')] <-
    predict(preproc, df)
```

¿Min-Max o Standardization?

Min-Max:

- Rango específico necesario
- Redes neuronales
- Algoritmos de imágenes
- No hay outliers extremos
- Sensible a outliers

Standardization:

- Distribución aproximadamente normal
- Regresión logística, SVM
- PCA, clustering
- Más robusto a outliers
- No acota valores

Regla práctica

¿En duda? $\rightarrow$ Usar Standardization

Importante: fit vs transform

Error común: Data leakage

Nunca usar estadísticas del conjunto de test para escalar

Flujo correcto

1. Dividir en train/test
2. `fit()` con datos de train (calcular min/max o media/std)
3. `transform()` train con parámetros calculados
4. `transform()` test con los MISMOS parámetros

Importante: fit vs transform

Error común: Data leakage

Nunca usar estadísticas del conjunto de test para escalar

Flujo correcto

1. Dividir en train/test
2. `fit()` con datos de train (calcular min/max o media/std)
3. `transform()` train con parámetros calculados
4. `transform()` test con los MISMOS parámetros

Código correcto

```
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Discretización (Binning)

¿Qué es?

Convertir variables continuas en categóricas dividiendo en intervalos

Ejemplo

Edad continua $\rightarrow$ Grupos de edad

Edad	Grupo
15	menor
25	joven
35	adulto
55	maduro
75	mayor

¿Por qué discretizar?

Ventajas

- Captura relaciones no lineales
- Más robusto a outliers
- Facilita interpretación
- Útil para análisis exploratorio
- Algunos modelos lo requieren

¿Por qué discretizar?

Ventajas

- Captura relaciones no lineales
- Más robusto a outliers
- Facilita interpretación
- Útil para análisis exploratorio
- Algunos modelos lo requieren

Desventajas

- Pérdida de información
- Introduce discontinuidades artificiales
- Elección de bins es subjetiva
- Puede empeorar performance

Tipos de discretización

1. Bins de igual ancho

Dividir el rango en intervalos iguales

```python
# Python
df['edad_grupo'] = pd.cut(
    df['edad'],
    bins=5,  # 5 intervalos iguales
    labels=['muy_joven', 'joven', 'adulto', 'mayor', 'muy_mayor']
)
```

```r
# R
df <- df %>%
  mutate(edad_grupo = cut(
    edad,
    breaks = 5,
    labels = c('muy_joven', 'joven', 'adulto', 'mayor', 'muy_mayor')
  ))
```

Bins con puntos específicos

```python
# Python - Bins personalizados
df['edad_grupo'] = pd.cut(
    df['edad'],
    bins=[0, 18, 30, 50, 65, 100],
    labels=['menor', 'joven', 'adulto', 'maduro', 'mayor']
)
```

```r
# R - Bins personalizados
df <- df %>%
  mutate(edad_grupo = cut(
    edad,
    breaks = c(0, 18, 30, 50, 65, 100),
    labels = c('menor', 'joven', 'adulto', 'maduro', 'mayor')
  ))
```

Bins de igual frecuencia (Quantiles)

Concepto

Cada bin contiene aproximadamente el mismo número de observaciones

```python
# Python - Cuartiles
df['ingreso_grupo'] = pd.qcut(
    df['ingreso'],
    q=4,  # 4 cuartiles
    labels=['bajo', 'medio_bajo', 'medio_alto', 'alto']
)
```

```r
# R - Cuartiles
df <- df %>%
  mutate(ingreso_grupo = cut(
    ingreso,
    breaks = quantile(ingreso, probs = seq(0, 1, 0.25), na.rm = TRUE
      ),
    labels = c('bajo', 'medio_bajo', 'medio_alto', 'alto'),
```

Combinar múltiples fuentes

¿Por qué integrar datos?

- Datos dispersos en múltiples tablas/archivos
- Enriquecer con información externa
- Análisis requiere combinar fuentes

Combinar múltiples fuentes

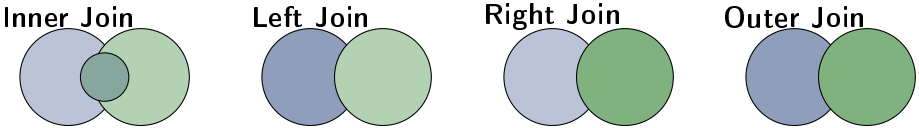
¿Por qué integrar datos?

- Datos dispersos en múltiples tablas/archivos
- Enriquecer con información externa
- Análisis requiere combinar fuentes

Dos operaciones principales

1. **Joins:** Combinar por columnas comunes (como SQL)
2. **Concatenación:** Apilar tablas vertical u horizontalmente

Tipos de Joins



- **Inner:** Solo filas con match en ambas
- **Left:** Todas de izquierda + matches de derecha
- **Right:** Todas de derecha + matches de izquierda
- **Outer:** Todas las filas de ambas

Joins en Python

```python
import pandas as pd

# Inner join
df_merged = pd.merge(df1, df2, on='id', how='inner')

# Left join
df_merged = pd.merge(df1, df2, on='id', how='left')

# Join con nombres diferentes
df_merged = pd.merge(
    df1, df2,
    left_on='cliente_id',
    right_on='id',
    how='left'
)

# Join con m ltiples columnas
```

Joins en R

```r
library(dplyr)

# Inner join
df_merged <- inner_join(df1, df2, by = "id")

# Left join
df_merged <- left_join(df1, df2, by = "id")

# Join con nombres diferentes
df_merged <- left_join(
  df1, df2,
  by = c("cliente_id" = "id")
)

# Join con m ltiples columnas
df_merged <- inner_join(
  df1, df2,
```

Concatenación

Python:

```python
# Vertical (apilar filas)
df_combined = pd.concat(
    [df1, df2],
    axis=0,              # 0 =
        vertical
    ignore_index=True # Resetear
        ndice
)

# Horizontal (a adir columnas)
df_combined = pd.concat(
    [df1, df2],
    axis=1               # 1 =
        horizontal
)
```

R:

```r
library(dplyr)

# Vertical
df_combined <- bind_rows(
  df1, df2
)

# Horizontal
df_combined <- bind_cols(
  df1, df2
)
```

Resumen de la clase

- **Normalización (Min-Max):** Escalar a [0, 1], sensible a outliers
- **Estandarización (Z-score):** Media 0, std 1, más robusto
- Usar `fit()` solo con train, `transform()` con train y test
- **Discretización:** Convertir continuas en categóricas
- Bins: igual ancho, personalizados, igual frecuencia
- **Joins:** Combinar tablas por columnas comunes
- Tipos: inner, left, right, outer
- **Concatenación:** Apilar vertical u horizontal

Próxima clase

APIs REST, web scraping y tidy data

Tarea

1. Usar el dataset transformado de la clase anterior
2. Normalización/Estandarización:
   - Aplicar Min-Max a variables numéricas
   - Aplicar Standardization a las mismas variables
   - Comparar distribuciones antes/después
3. Discretización:
   - Elegir 2 variables continuas
   - Crear bins con diferentes estrategias
   - Visualizar resultados
4. Si tenés múltiples tablas, practicar joins
5. Documentar todo en el notebook
6. Subir a GitHub

¿Preguntas?